

claude-windows / fa4dc4eb-4705-47fd-b044-4f017b586b15

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\fa4dc4eb-4705-47fd-b044-4f017b586b15\subagents\workflows\wf_ac0cbb29-6bf\agent-aa867fcea312d681c.jsonl`  
- SHA-256: `fad06fbe4b866c1c5be3ab33cba8d2e8ecd76485eba1652ad37cfb2d180199a6`  
- Source modified: `2026-06-19T15:53:27+00:00`  
- Imported at: `2026-07-05T16:48:22+00:00`  
- project: `wf_ac0cbb29-6bf`  
- session_id: `fa4dc4eb-4705-47fd-b044-4f017b586b15`
```

Transcript

1. user (2026-06-19T15:52:06.174Z)

You are auditing a badminton rally-detection codebase for DOC/COMMENT STALENESS ? statements that CONTRADICT the current code or data.

HARD RULES:

- Report ONLY factual staleness with CODE EVIDENCE (file:line or symbol) that proves it. Open the cited code and confirm before reporting.
- NEVER invent a fact or a status. If a thing is built-but-default-OFF / unwired / unmeasured, say EXACTLY that ? do not overclaim "in production".
- A "[design]" tag is stale ONLY if the thing is actually BUILT in code (cite the symbol).
- DO NOT propose subjective rewrites, style/tone changes, or improvements. DO NOT relitigate owner-gated DECISIONS (owned-model roadmap, licensing/MIT-Apache stance, product/strategy choices, genuinely-still-design proposals).
- Preserve the document's voice and formatting; make the SMALLEST surgical edit. stale_text MUST be copied verbatim with enough context to be unique.
- IGNORE docs/archives/** (terminal snapshots) and any memory/ file.

Working dir is the repo root; use Read/Grep/Glob freely.

ADVERSARIALLY VERIFY these proposed staleness fixes for ONE file: docs/CODE_MAP.md.

For EACH proposed fix: (1) open docs/CODE_MAP.md and confirm stale_text appears VERBATIM (tighten/expand it to an exact, UNIQUE match if needed ? this text will be applied by exact string match, so it must be perfect); (2) re-open the cited code yourself and confirm the claim is genuinely contradicted by the CURRENT code; (3) confirm corrected_text is accurate, preserves voice, and is minimal; (4) DROP it if it is subjective, an owner-gated decision, already-correct, or you cannot verify the evidence. Output ONLY the survivors (verdict confirm or revise), with stale_text guaranteed to be an exact unique substring of the file.

PROPOSED FIXES:

```
[  
  {  
    "file": "docs/CODE_MAP.md",  
    "stale_text": "`db.create_job` -> `_get_executor().submit(_run_job,...)` -> **202 +  
job_id**. `::_run_job` (worker: mark running -> `_execute_processing` -> done/failed; catches +  
logs traceback).",  
    "corrected_text": "`db.create_job` -> `_get_executor().submit(_drain_due_jobs)` -> **202 +  
job_id**. `::_drain_due_jobs` / `::_run_claimed_job` (worker loop EXTRACTED to  
`@backend/api/job_worker.py` (#234), re-exported on `backend.main`): claim each runnable job via  
`db.claim_due_job` -> `_execute_processing` -> mark done/failed (a `JobWaiting` parks the job  
via `set_job_waiting` for resume; EXECUTION_POLICY P3 self-scheduling).",  
    "why_stale": "`_run_job` no longer exists in the code. `backend/main.py::process_video`  
(line 264-267) now does `db.create_job(...)` then `_get_executor().submit(_drain_due_jobs)`, not  
`submit(_run_job,...)`. The per-job worker is `_run_claimed_job` and the drain loop is  
`_drain_due_jobs`, both now in `backend/api/job_worker.py` (#234) and re-exported on  
`backend.main` (main.py lines 71-75). `_run_job` appears only in CODE_MAP.md,  
EXECUTION_POLICY.md and a test name (grep over the repo).",  
    "severity": "high"
```

```

    },
    {
        "file": "docs/CODE_MAP.md",
        "stale_text": "`::freeze_synthetic` (3 committed synthetic Tier-0 fixtures),
`::freeze_real_fixture` (P1/M2: `RefusalError` unless `owned`+`minors_free`+non-blank
`license`;)",
        "corrected_text": "`::freeze_synthetic` (3 committed synthetic Tier-0 fixtures),
`freeze_real_fixture` (now DEFINED in the split-out `@backend/eval/golden_real_fixtures.py`
(#233) and re-exported here via PEP-562 `__getattr__`) (P1/M2: `RefusalError` unless
`owned`+`minors_free`+non-blank `license`;)",
        "why_stale": "Per #233 `freeze_real_fixture` and `RefusalError` were moved OUT of
golden_fixtures.py into the new module `backend/eval/golden_real_fixtures.py` (defined there:
`RefusalError` line 59, `freeze_real_fixture` line 157). golden_fixtures.py only re-exports them
lazily (`_REAL_FIXTURE_EXPORTS`, lines 822-834). CODE_MAP §2.3 still lists
`::freeze_real_fixture` as a symbol of golden_fixtures.py, and the new golden_real_fixtures.py
module is entirely absent from the §2.3 map.",
        "severity": "medium"
    }
]

```

2. assistant (2026-06-19T15:52:08.707Z)

I'll verify each proposed fix against the current code and the CODE_MAP.md file.

3. user (2026-06-19T15:52:12.195Z)

fatal: Cannot fast-forward to multiple branches.

4. user (2026-06-19T15:52:12.342Z)

114:[Omitted long matching line]

5. user (2026-06-19T15:52:12.471Z)

183:[Omitted long matching line]

6. assistant (2026-06-19T15:52:14.440Z)

Let me read the relevant sections of CODE_MAP.md and the cited code files.

7. user (2026-06-19T15:52:15.948Z)

```

108
109     ## 2. SUBSYSTEMS
110
111     ### 2.0 Orchestration & config
112     - `@backend/main.py` ? Thin CLI + FastAPI bootstrap (orchestrator; modes via FLAGS).
Static UI mounts `/static` from `backend/api/static`. Logging is initialized with `basicConfig`
(INFO, timestamped format); backend modules use `logger = logging.getLogger(__name__)` (replaces
stray prints in hot paths like motion, stitcher, wasb_infer, config loader). User-facing CLI
summaries may still use `print` for direct output.
113     - `::app` FastAPI; `::db_instance`, `::global_config` (`Optional[AppConfig]`) module
globals set ONLY in `main()`. ? importing `app` for ASGI without `main()` -> both `None` ->
every endpoint NPES.
114     - **ASYNC (#16, PR-1):** `::process_video` `@POST /api/process` -> fast-fail 4xx
(path/exists/segmenter) -> `db.create_job` -> `_get_executor().submit(_run_job,...)` -> **202 +
job_id**. `::run_job` (worker: mark running -> `_execute_processing` -> done/failed; catches +
logs traceback). `::execute_processing` = the former sync body (hash -> validate -> add_video
-> segment -> `finally:` always `emit_indexer_report`, grafts `response["reports"]`; never
raises from the report); a RAISING segmenter now prune_after-restores pre-run rows before re-
raising. `::get_job` `@GET /api/jobs/{job_id}` -> `{status, progress, video_id, error, result,
reports, ...}` ? job `status` = EXECUTION state (`queued|running|done|failed`); the pipeline
verdict lives in `result["status"]`. `::get_executor` lazy module-global
`ThreadPoolExecutor(max_workers=1)` (single box + Gemini serial-upload learning; tests

```

```

monkeypatch it inline). `main()` runs `db.fail_stale_jobs()` at startup (orphaned queued/running
-> failed). `::compile_highlights` `@POST /api/compile`. `::query_play_segments` `@POST
/api/query`. `::get_config` `@GET /api/config`. `::run_cli_diagnose_clip` `--debug-clip`: lazy
`import cv2`, samples ±3s vs `indexing.motion_threshold` (dual model/dict read). ? CLI `--video-
path` mode still runs the pipeline synchronously inline (separate code path, unchanged by #16).
115 - **UPLOAD/DOWNLOAD (PR-2, B2+B3):** the chunked-upload endpoints now live in
`@backend/api/uploads.py` (#228 ? an `APIRouter` registered via `app.include_router`;
`uploads.py::upload_cfg` resolves the live `backend.main.global_config` lazily so it reads the
startup config, and a lazy `import backend.main` avoids an import cycle) ?
`uploads.py::upload_init` `@POST /api/upload/init` (`{filename, total_size_bytes?}` -> ext
allowlist + size gate -> `{upload_id}`), `::upload_part` `@PUT /api/upload/{id}/part/{n}` (raw-
bytes body, STRICTLY sequential n, streams to `.partial-<id>` under `security.upload_dir`; per-
part/total cap breach -> 413 + upload ABORTED), `::upload_complete` `@POST ../complete`
(`{total_parts}` -> assemble -> never-overwrite rename -> `{path, video_id(MD5), size_bytes}` ?
client then POSTs /api/process with that path), `::upload_abort` `@DELETE /api/upload/{id}`. ?
upload state (`uploads.py::uploads` + `::uploads_lock`) is IN-PROCESS ? restart aborts
partials (client re-inits); per-user partitioning lands with PR-3. `main.py::download_highlight`
`@GET /api/highlights/{video_id}/{filename}` (download was NOT moved ? still in main.py) ? safe-
name + `resolve_under_root(output_dir)` then `FileResponse` stream (the old /api/compile alert
path was browser-unreachable). Config: `SecurityConfig.{upload_dir='output/uploads',
max_upload_mb=4096, max_part_mb=95, allowed_upload_extensions=[mp4,mov]}` (95MB: free-tier edge
proxies cap bodies ~100MB).
116 - `::load_config` ? ? legacy thin wrapper returning a **dict**
(`load_app_config(path).model_dump(...)`); kept for transition ? new code imports
`load_app_config`/`AppConfig` directly. `::ProcessRequest`/`::QueryRequest`/`::CompileRequest`
pydantic bodies.
117 - `main()` sets `global_config.output.output_dir = args.output_dir` (typed field on
`OutputConfig`); `db_path = os.path.join(global_config.output.output_dir, output.db_name)`.
`compile_highlights` reads the SAME `global_config.output.output_dir`, so `--output-dir` is
honored end-to-end (the old write-only `runtime_overrides` hack is gone).
118 - ? `--segmenter` argparse `choices` frozen at build time from
`segmenter_registry.list_available()` ? a lazily-registered segmenter wouldn't appear. CLI
default_segmenter fallback `motion`.
119 - `@scripts/run_process_and_stitch.py` ? one-shot demo; ? hardcoded paths, interactive
`input()` (blocks automation), skips validation. Config via `backend.config.load_config`
(typed); segmenter, padding, and db path all read from the model (no literal fallbacks).

```

8. user (2026-06-19T15:52:16.623Z)

```

178 - `@backend/eval/distill_local.py` ? distill Gemini -> a LEARNED classifier (beyond
calibrate_local's threshold ceiling). `::FEATURE_NAMES` (5 fps/res-INVARIANT:
duration,peak,mean,active_ratio,net_crossings), `::build_candidates` (recall-first proposal),
`::predict_rallies` (proba filter then `gemini_refine.merge_overlaps(gap_tol=1.0)`),
`::run`/`::main`. consts `PROPOSAL=(0.005,1.0,0.5)`, `BASELINE_LOCAL=(0.02,2.0,2.0)`. ? leave-
one-video-out needs ?2 videos; final model trained on all.
179 - `@backend/eval/gemini_refine.py` ? refine WASB candidates with Gemini over SHORT
padded clips (precision lever; eval/tuning, NOT the live pipeline).
`::merge_overlaps(gap_tol=0.0)` (reused by distill_local with gap_tol=1.0), `::refine_window` (?
FRESH genai client per worker ? shared throws socket errors), `::full_video_rallies`,
`::run`/`::main` (`--full-video` skips `--trajectory`). ? requires `GEMINI_API_KEY` (raises
SystemExit); imports `BASELINE` from calibrate_wasb +
`TUNED`/`write_segments_csv`/`seconds_to_mms` from export_wasb_segments.
180 - `@backend/eval/regression.py` ? CI gate. `::DEFAULT_TOLERANCE=0.05`,
`::DEFAULT_BASELINE_PATH=eval_baselines/regression_baseline.json` (tracked, NOT under gitignored
output/), `::regress`, `::regress_with_provider` (GT via ?`annotation_registry.get(tier)`;
"file"->`FileProvider`/"vendor"->`HumanVendor`/"auto"->`oracle`),
`::save_baseline`/`::load_baselines`, `::gt_hash` (label-set fingerprint, reused by
experiment.py), `::RegressionResult`. ? imports `annotation_registry` from
**`backend.annotations`** NOT `backend.eval.annotations`; gate fires on precision OR recall drop
(F1 reported, not gated); no baseline -> `regressed=False` (silent pass). CLI
`run_regression.py` exit 0 ok / 1 regress / 2 no-DB / 3 no-baseline = the promotion gate.
181 - `@backend/eval/experiment.py` ? A/B + shadow + **SxS** variant layer (the thin
"experiment" glue per `docs/EXPERIMENT_HARNESS.md`; composes
metrics/calibration/classifier/training/regression ? NO new scoring math).
`::Variant` (segmenter,config_overrides,cue_flags,`min_crossings`[Gate A],`min_players`[Gate

```

```

B],classifier_model,feature_names,threshold,merge_gap;
`to_dict/from_dict/spec_hash/is_learned`), `::CONTROL` (pinned zero-gate baseline).
`::VideoCandidates` (name,intervals,features{col-major},gts), `::predict` (gate?learned-
filter?`gemini_refine.merge_overlaps`), `::evaluate_variant` (static / pinned-model / **honest-
LOVO** train-on-others ? mirrors distill_local), `::compare` (labeled **B?A** deltas +
`label_set_hash`), `::compare_unlabeled` (**NEW-footage SxS**: agreed/a_only/b_only via
`match_intervals`, no GT ? the owner's "SxS on new videos"), `::record_experiment` (?
`eval_baselines/experiment_leaderboard.json`), `::load_video_candidates` (DB bridge via
`query_candidate_segments`). ? enabling a gate needs its feature persisted (absent?0.0?fails
gate, warns); learned variant on an unlabeled video needs a pinned `classifier_model`. **No-reg
invariant (tested): all cues OFF ? byte-identical to CONTROL.**
182
183     - `@backend/eval/golden_fixtures.py` ? **video-free regression harness**
(`docs/GOLDEN_REGRESSION_FIXTURES.md`). `::replay_fixture` =
`tracknet_runner.parse_trajectory_csv` ? `distill_local.build_candidates` (PINNED preset) ?
`experiment.VideoCandidates` (5 shuttle cols only) ? `experiment.predict(CONTROL)` ?
`metrics.score`; **CONTROL / pure-stdlib ? no numpy on the replay path** (cross-OS stable; the
learned/numpy path is deliberately excluded). `::freeze_synthetic` (3 committed synthetic Tier-0
fixtures), `::freeze_real_fixture` (P1/M2: `RefusalError` unless `owned`+`minors_free`+non-blank
`license`; opaque random `fx<hex>` slug, metric-invariant time-origin reset, STRICT
`gt_loader`, positive no-`FORBIDDEN_KEYS`/MD5/source-path scan), `::refresh_snapshot`,
`::compare_expected` (exact-int / approx-1e-6, NEVER byte-compare),
`::load_manifest`/`::load_fixture` (schema_version dispatch). CLI `--freeze-
synthetic`/`--freeze-real`/`--refresh-snapshot`/`--check`. Tests:
`tests/test_golden_fixtures.py` (M1/M3 characterization) + `tests/test_golden_real_fixtures.py`
(P1/M2). CI: the `golden-crossos` job (ubuntu/windows/macOS) runs `--check` + the M1 test. ?
synthetic-tier numbers = plumbing correctness, NOT field quality.
184
185     ### 2.3a Audio (E1 probe; NOT adopted for fusion ? ADR-007)
186     - `@backend/pipeline/audio/hit_detector.py` ? classical-onset shuttle-hit detector. Pure
numpy + stdlib `wave` (NO scipy); ffmpeg only for extraction. `::HitEvent` (t,strength),
`::extract_audio` (ffmpeg -> mono 16k PCM WAV), `::load_wav`, `::onset_envelope` (pure-numpy
STFT spectral flux; `band=(lo,hi)` band-limit), `::detect_hits` (adaptive `mean+k.std`;
`k`=sensitivity knob), `::detect_hits_in_video`. ? separate signal ? WASB never modified by
this; per audio-e1-findings recall 100% but discrimination ~2.2x, band-pass null -> NOT adopted
(PR #35).
187     - `@backend/eval/audio/e1_probe.py` ? E1 GO/NO-GO diagnostic (no training/fusion).
`::cluster_hits` (groups thwacks; ? `min_hits>=2` so a lone service-feed hit isn't a rally),
`::run` (discrimination ratio + recall recovery + audio-only P/R/F1), `::main`. reuses
`hit_detector` + `calibrate_wasb.{load_golden_gts,make_predict_fn,BASELINE}` + `metrics`.
188
189     ### 2.4 Stitcher / tools / utils

```

9. assistant (2026-06-19T15:52:19.471Z)

Now let me verify the git branch state and check the cited code files. First the job worker claim.

10. user (2026-06-19T15:52:21.213Z)

```

docs/prominent-court-design
3f5a56c fix(gemini): surface failed chunks loudly (no more silent partial reels) (#240)
eff12c0 chore(eval): re-baseline the nightly regression to the n=15 golden corpus (#239)
a3b4954 feat(runner): --min-rally-duration flag (surface the long-rally reel filter on the CLI)
(#238)
a240221 fix(test): ablation litmus uses canonical experiment._is_fusion_schema (unbreaks the
suite) (#237)
d749fb9 test(eval): re-ground served Gate-A litmus on the present (multicourt) golden set (#236)

```

11. user (2026-06-19T15:52:22.455Z)

```

66:# `backend.main` (`JobWaiting`, `_drain_due_jobs`, `_arm_wake_timer`, `_get_executor`,
67:# `_run_claimed_job`, `_MAX_DRAIN_WAKE_SEC`, ...). The worker resolves these seams THROUGH
74:     _drain_due_jobs,

```

```

75:     _get_executor,
78:     _run_claimed_job,
247:     worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
264:     if not db_instance.create_job(job_id, req.video_path, req.segmenter,
267:     _get_executor().submit(_drain_due_jobs)
637:     _get_executor().submit(_drain_due_jobs)

```

12. user (2026-06-19T15:52:22.986Z)

backend\api\job_worker.py

13. user (2026-06-19T15:52:23.080Z)

backend\eval\golden_real_fixtures.py

14. assistant (2026-06-19T15:52:26.808Z)

Both new module files exist. Now let me read the cited code in detail to confirm each claim.

15. user (2026-06-19T15:52:28.259Z)

```

60     # Global variables initialized at startup
61     db_instance: Optional[Database] = None
62     global_config: Optional[AppConfig] = None # now typed (Pydantic model)
63
64     # Async job worker ? drain/wake loop (DEFERRED_HARDENING_PLAN #16, EXECUTION_POLICY.md
65     P3)
66     # now lives in backend.api.job_worker. Re-exported here so the names stay addressable on
67     # `backend.main` (`JobWaiting`, `_drain_due_jobs`, `_arm_wake_timer`, `_get_executor`,
68     # `_run_claimed_job`, `_MAX_DRAIN_WAKE_SEC`, ...). The worker resolves these seams
69     THROUGH
70     # this module object at call-time, so `monkeypatch.setattr(backend.main, ...)` still
71     # intercepts the inter-function calls (a parked job arms the PATCHED `_arm_wake_timer`,
72     # so no real daemon timer leaks in tests). Behavior is unchanged by the extraction.
73     from backend.api.job_worker import ( # noqa: F401 (re-exports: names stay addressable
74     on backend.main)
75     JobWaiting,
76     _arm_wake_timer,
77     _drain_due_jobs,
78     _get_executor,
79     _job_to_request,
80     _MAX_DRAIN_WAKE_SEC,
81     _run_claimed_job,
82     _schedule_next_drain_if_waiting,
83     )
84
85     # Legacy thin wrapper for any remaining direct calls during transition.
86     # New code should import load_app_config / AppConfig from backend.config directly.
87     def load_config(config_path: str = "config.json") -> Dict[str, Any]:

```

16. user (2026-06-19T15:52:28.710Z)

```

240     @app.post("/api/process", status_code=202)
241     def process_video(req: ProcessRequest):
242         """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.
243
244         Fast-fail checks (bad path, missing file, unknown segmenter) happen here so the
245         client gets an immediate 4xx; everything slow ? including MD5 hashing of the file ?
246         runs on the worker (DEFERRED_HARDENING_PLAN #16). The row is created 'queued'; the
247         worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
248         """
249         allowed_root = getattr(getattr(global_config, "security", None),
250         "allowed_video_root", None)
251         try:

```

```

251         validate_input_path(req.video_path, allowed_root=allowed_root)
252     except ValueError as e:
253         raise HTTPException(status_code=400, detail=str(e)) from e
254     if not os.path.exists(req.video_path):
255         raise HTTPException(status_code=404, detail=f"Video file not found at
'{req.video_path}'")
256     if req.segmenter and not segmenter_registry.get(req.segmenter):
257         available = list(segmenter_registry.list_available().keys())
258         raise HTTPException(
259             status_code=400,
260             detail=f"Segmenter '{req.segmenter}' not found. Available segmenters:
{available}"
261         )
262
263     job_id = uuid.uuid4().hex
264     if not db_instance.create_job(job_id, req.video_path, req.segmenter,
265                                 req.player_pool, req.max_frames):
266         raise HTTPException(status_code=500, detail="Failed to persist job record.")
267     _get_executor().submit(_drain_due_jobs)
268     return JSONResponse(
269         status_code=202,
270         content={"job_id": job_id, "status": "queued", "poll": f"/api/jobs/{job_id}"},
271     )
272
273
274     @app.get("/api/jobs/{job_id}")

```

17. assistant (2026-06-19T15:52:34.489Z)

Fix #1 is confirmed: `process\_video` now does `db.create\_job(...)` then `\_get\_executor().submit(\_drain\_due\_jobs)`, and `\_run\_claimed\_job`/`\_drain\_due\_jobs` are imported from `backend.api.job\_worker`. Let me verify the job_worker module internals to confirm the corrected_text details (claim_due_job, set_job_waiting, JobWaiting parking).

18. user (2026-06-19T15:52:36.500Z)

```

4:The worker DRAINS the jobs table via `claim_due_job` and honours WAIT_AND_RESUME: a job
5:that raises `JobWaiting` is parked (status 'waiting' + `next_poll_at` + progress) and
9:every cross-function seam ? `db_instance`, `global_config`, `_execute_processing`,
41:class JobWaiting(Exception):
44:     NOT a failure. The worker persists `next_poll_at` + `progress` (via `set_job_waiting`),
45:     releases the thread, and re-claims the job when due (`claim_due_job`). No work to date
46:     is the segmenter's `_execute_processing`, run UNDER the destroy-after-confirm contract.
61:     re-claimed by `claim_due_job` (queued OR resumed-from-waiting) runs identically to
72:def _run_claimed_job(job: Dict[str, Any]) -> None:
73:     """Run ONE job already claimed (`status='running'`) by `claim_due_job`, recording its
74:     terminal state ? OR parking it for resume on `JobWaiting` (WAIT_AND_RESUME).
86:     result = _main._execute_processing(
91:     except _main.JobWaiting as w:
94:         db_instance.set_job_waiting(job_id, w.delay_sec, progress=w.progress)
100:def _drain_due_jobs() -> int:
104:     Each parked job (`JobWaiting` ? status 'waiting') drops out of this drain and is picked
112:     job = _main.db_instance.claim_due_job()

```

19. user (2026-06-19T15:52:36.646Z)

Found 3 files
docs\CODE_MAP.md
tests\test_job_worker_loop.py
docs\EXECUTION_POLICY.md

20. assistant (2026-06-19T15:52:41.069Z)

Fix #1 fully confirmed. `\_run\_job` no longer exists in production code (only in CODE_MAP.md,

EXECUTION_POLICY.md, and a test name). The worker is `\_drain\_due\_jobs`/`\_run\_claimed\_job` in `backend/api/job\_worker.py`, using `claim\_due\_job` and `set\_job\_waiting`/`JobWaiting`. Let me check the test name to ensure "test name" claim is accurate, then verify the #234 PR reference and re-export details, and confirm fix #2.

21. user (2026-06-19T15:52:42.006Z)

7:due ? instead of the old one-shot `submit(\_run\_job, ...)`.
87: # Submit via the public endpoint (which now submits a drain tick, not _run_job).

22. user (2026-06-19T15:52:43.462Z)

```
1      """Golden Regression Fixtures (P1 + M2) ? de-attribution / minimization + the refusal
gate.
2
3      This module holds the real-fixture subsystem split out of ``golden_fixtures`` (which
keeps the
4      shared synthetic/replay primitives). It turns an OWNED, minors-free real trajectory +
golden labels
5      into a committable, de-attributed Tier-0 fixture, reusing M1's ``replay_fixture`` for
the snapshot.
6      (docs/GOLDEN_REGRESSION_FIXTURES.md §4c anti-attribution, §5 threat model, §7 row P1, §6
honest
7      limits.)
8
9      The shared primitives (constants, float/JSON helpers, ``replay_fixture``, the CSV
writers, the
10     manifest appender) live in ``golden_fixtures`` and are imported FROM it here ? there is
a single
11     source of truth, no duplicated logic/thresholds. ``golden_fixtures`` re-exports the
public names
12     defined here (``freeze_real_fixture``, ``RefusalError``, ``_scan_forbidden``) at the
bottom of that
13     module so every historical ``golden_fixtures.<name>`` import path stays valid.
14
15     §6 honest-limits caveat: a green suite proves only that the deterministic post-
trajectory transforms
16     are unchanged for the frozen cases ? characterization is behaviour-locking, NOT
correctness.
17     """
18     from __future__ import annotations
19
20     import os
```

23. user (2026-06-19T15:52:44.823Z)

```
59:class RefusalError(ValueError):
157:def freeze_real_fixture(
```

24. user (2026-06-19T15:52:45.661Z)

```
791:      # The real-fixture subsystem (P1 + M2) lives in golden_real_fixtures (split out);
import it
792:      # here so the CLI reaches the SAME freeze_real_fixture / RefusalError it always has.
793:      from backend.eval import golden_real_fixtures
801:      expected = golden_real_fixtures.freeze_real_fixture(
808:      except golden_real_fixtures.RefusalError as e:
823:# Re-exports ? the real-fixture subsystem (P1 + M2) lives in ``golden_real_fixtures`` but
827:#: The names the real-fixture module owns; re-exported lazily (PEP 562 ``__getattr__``) so
the
828:#: ``golden_fixtures`` ? ``golden_real_fixtures`` cycle resolves regardless of import order
?
829:#: ``golden_real_fixtures`` imports the shared primitives FROM this module, so an EAGER
bottom-of-
```

```
830:#: file re-export would fail when ``golden_real_fixtures`` is imported FIRST (its own names
would
833:_REAL_FIXTURE_EXPORTS: Tuple[str, ...] = (
834:     "RefusalError", "freeze_real_fixture", "_scan_forbidden", "_new_surrogate_slug",
839:def __getattr__(name: str) -> Any:
840:     """Lazily resolve the re-exported real-fixture names from ``golden_real_fixtures`` (PEP
562)."""
841:     if name in _REAL_FIXTURE_EXPORTS:
842:         from backend.eval import golden_real_fixtures
843:         return getattr(golden_real_fixtures, name)
```

25. assistant (2026-06-19T15:52:48.465Z)

Both fixes are confirmed. Now let me verify the PR number references (#234, #233) and confirm the exact line numbers cited in the proposed fixes, plus verify the stale_text for fix #2 is a unique match (the line has two `freeze\_real\_fixture` mentions ? let me check uniqueness).

26. user (2026-06-19T15:52:50.658Z)

```
eff12c0 chore(eval): re-baseline the nightly regression to the n=15 golden corpus (#239)
a3b4954 feat(runner): --min-rally-duration flag (surface the long-rally reel filter on the CLI)
(#238)
a240221 fix(test): ablation litmus uses canonical experiment._is_fusion_schema (unbreaks the
suite) (#237)
d749fb9 test(eval): re-ground served Gate-A litmus on the present (multicourt) golden set (#236)
5f555cf refactor(eval): extract one shared golden-feature loader behind the three load_videos
(#235)
5409fc4 refactor(api): extract job-worker drain/wake loop from main into backend.api.job_worker
(#234)
5c46ebc refactor(eval): split real-fixture subsystem into golden_real_fixtures (pure move)
(#233)
97fa187 refactor(db): route bool-returning single-statement writers through _execute_write
(#232)
ae10447 refactor(eval): extract _build_preset(args, side) from rally_seg_eval.main() (#231)
1e324ea refactor(motion): extract detection + DB-populate halves from process_video (#230)
174cef6 refactor(api): extract job-worker drain/wake loop from main into backend.api.job_worker
727a421 refactor(eval): split real-fixture subsystem into golden_real_fixtures (pure move)
3194a27 fix(review): address #157 owner feedback - update living-doc process rule to current
convention (author loop fixes, owner merges after LGTM); relabel
test_raising_segmenter_triggers_restore_contract + point to real coverage in reingest_safety +
job_worker_loop tests; fix bare-list duck-typing to exactly mirror main.py expression; correct
#156 merge type in doc (merge commit, not squash)
```

27. user (2026-06-19T15:52:51.672Z)

```
183:[Omitted long matching line]
```

28. assistant (2026-06-19T15:52:54.374Z)

PR #234 (job_worker extraction) and #233 (golden_real_fixtures split) are both confirmed in git history. Both fixes' PR references are accurate.

Now let me confirm the exact stale_text strings are unique substrings of the file. Let me check the exact byte content for both.

29. user (2026-06-19T15:52:58.293Z)

```
s1 count: 1
s2 count: 1
```

30. assistant (2026-06-19T15:53:07.295Z)

Both stale_text strings are exact unique substrings of the file. Both fixes are fully verified:

****Fix #1 (high):**** Confirmed. ``backend/main.py:264-267`` does ``db_instance.create_job(...)`` then ``_get_executor().submit(_drain_due_jobs)`` ? not ``submit(_run_job,...)``. ``_run_job`` no longer exists in production code (only in `CODE_MAP.md`, `EXECUTION_POLICY.md`, and a test docstring/name). The per-job worker ``_run_claimed_job`` and drain loop ``_drain_due_jobs`` live in ``backend/api/job_worker.py`` (#234), re-exported on ``backend.main`` (`main.py:71-79`). The ``JobWaiting`/`set_job_waiting`/`claim_due_job`` parking flow is confirmed in ``job_worker.py``.

****Fix #2 (medium):**** Confirmed. ``RefusalError`` (`golden_real_fixtures.py:59`) and ``freeze_real_fixture`` (`golden_real_fixtures.py:157`) are now `DEFINED` in ``backend/eval/golden_real_fixtures.py`` (#233), with ``golden_fixtures.py`` re-exporting them lazily via `PEP-562`__getattr__`` (``_REAL_FIXTURE_EXPORTS``, lines 833-843). `CODE_MAP §2.3` still lists ``::freeze_real_fixture`` as a symbol of ``golden_fixtures.py`` and the new module is absent.

I have a minor accuracy note on fix #2's `corrected_text`: it cites the re-export lines as "822-834" but the actual ``_REAL_FIXTURE_EXPORTS`` tuple is at line 833 and ``__getattr__`` at 839 ? however the `corrected_text` itself does not mention line numbers (only the `why_stale` does, which isn't applied to the file), so the `corrected_text` stands as-is. I'll return it as a confirm.

31. user (2026-06-19T15:53:23.638Z)

Structured output provided successfully

32. assistant (2026-06-19T15:53:27.593Z)

Both proposed fixes survived adversarial verification and are returned as ``confirm``.